



TERM PROJECT REPORT
MINHAJ UNIVERSITY OF LAHORE
DEPARTMENT OF COMPUTER SCIENCE
SEMESTER (6thD)
THREE ADDRESS CODE (TAC)

Submitted

Ali
Amna
Ayesha
Hareem

By

Ahmad
Afzal
Hussain

Submitted to

Sir
Subject: Compiler Construction

Rashid

Abstract

Three Address Code (TAC) is an intermediate representation used in compiler construction to simplify complex programming expressions into a sequence of simple instructions. Each instruction contains at most three addresses, including two operands and one result. TAC plays a significant role in code optimization and target code generation. It acts as a bridge between the front-end and back-end phases of a compiler, making program translation more efficient and machine-independent.

Problem Statement

High-level programming languages contain complex expressions that are difficult to optimize and directly convert into machine code. Compilers require an intermediate representation that simplifies these expressions while preserving their meaning. Three Address Code (TAC) addresses this problem by breaking complex operations into smaller and simpler instructions, enabling efficient optimization and code generation.

Introduction

Compiler construction is the process of translating source code written in a high-level programming language into machine-understandable code. During compilation, an intermediate representation is generated to simplify the translation process.

Three Address Code (TAC) is one of the most widely used intermediate representations. It transforms complex expressions into simple instructions containing at most three addresses. TAC improves compiler efficiency, supports optimization techniques, and simplifies machine code generation.

Objectives

- Understand the concept of Three Address Code.
- Study the role of TAC in compiler construction.
- Learn the workflow of TAC generation.
- Analyze different TAC representations.
- Understand the advantages and limitations of TAC.

What is Three Address Code?

Three Address Code is an intermediate code representation in which each instruction contains at most three addresses:

- Operand 1
- Operand 2
- Result

General Form

Result = Operand1 Operator Operand2

Example

Expression:

$a = (b - c) + (b - c)$

TAC Representation:

$t1 = b - c$

$t2 = b - c$

$t3 = t1 + t2$

$a = t3$

Where $t1$, $t2$, and $t3$ are temporary variables.

Workflow of Three Address Code

The workflow of TAC follows the compiler phases:

Source Program

↓

Lexical Analysis

↓

Syntax Analysis

↓

Semantic Analysis

↓

Intermediate Code Generation (TAC)



Code Optimization

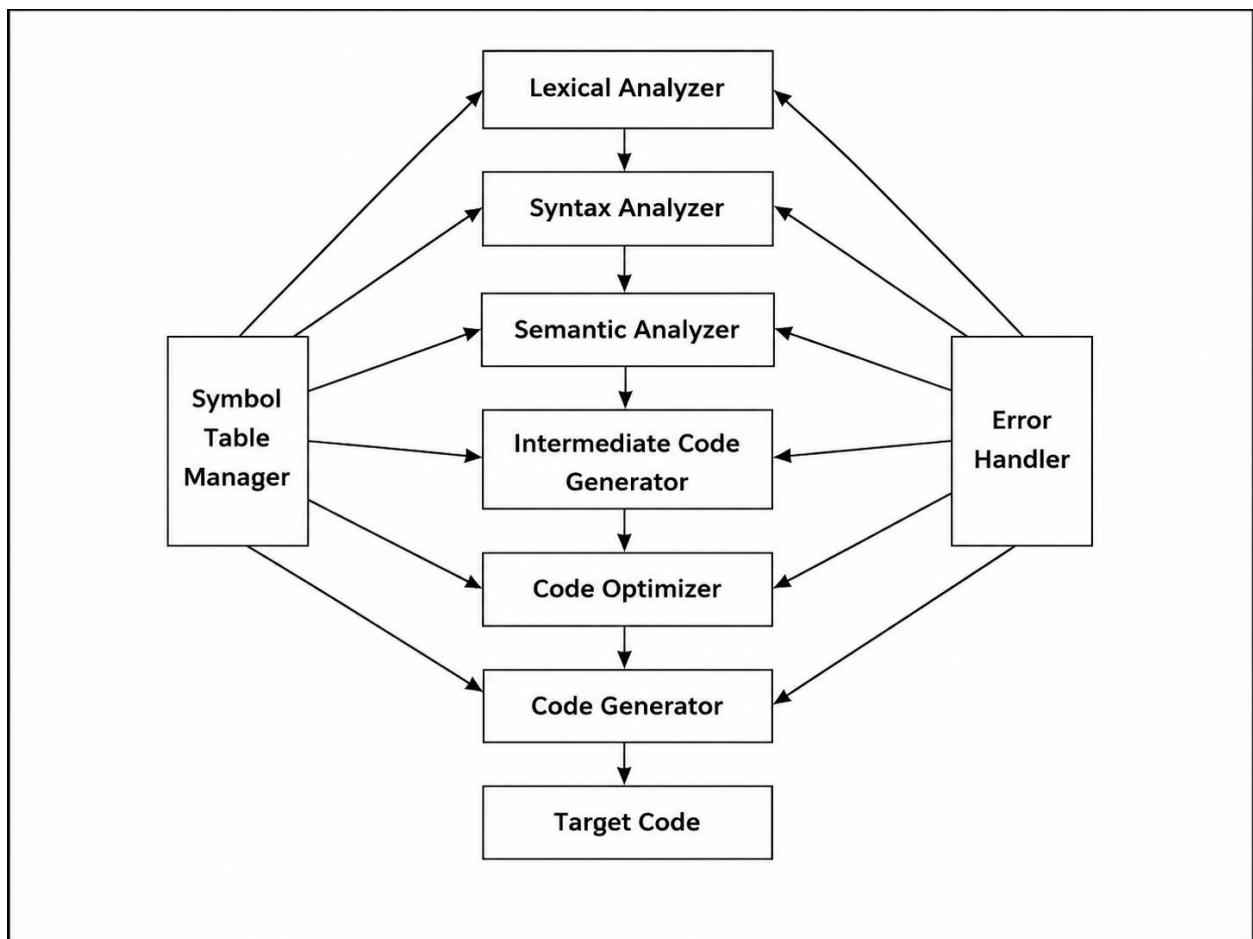


Target Code Generation



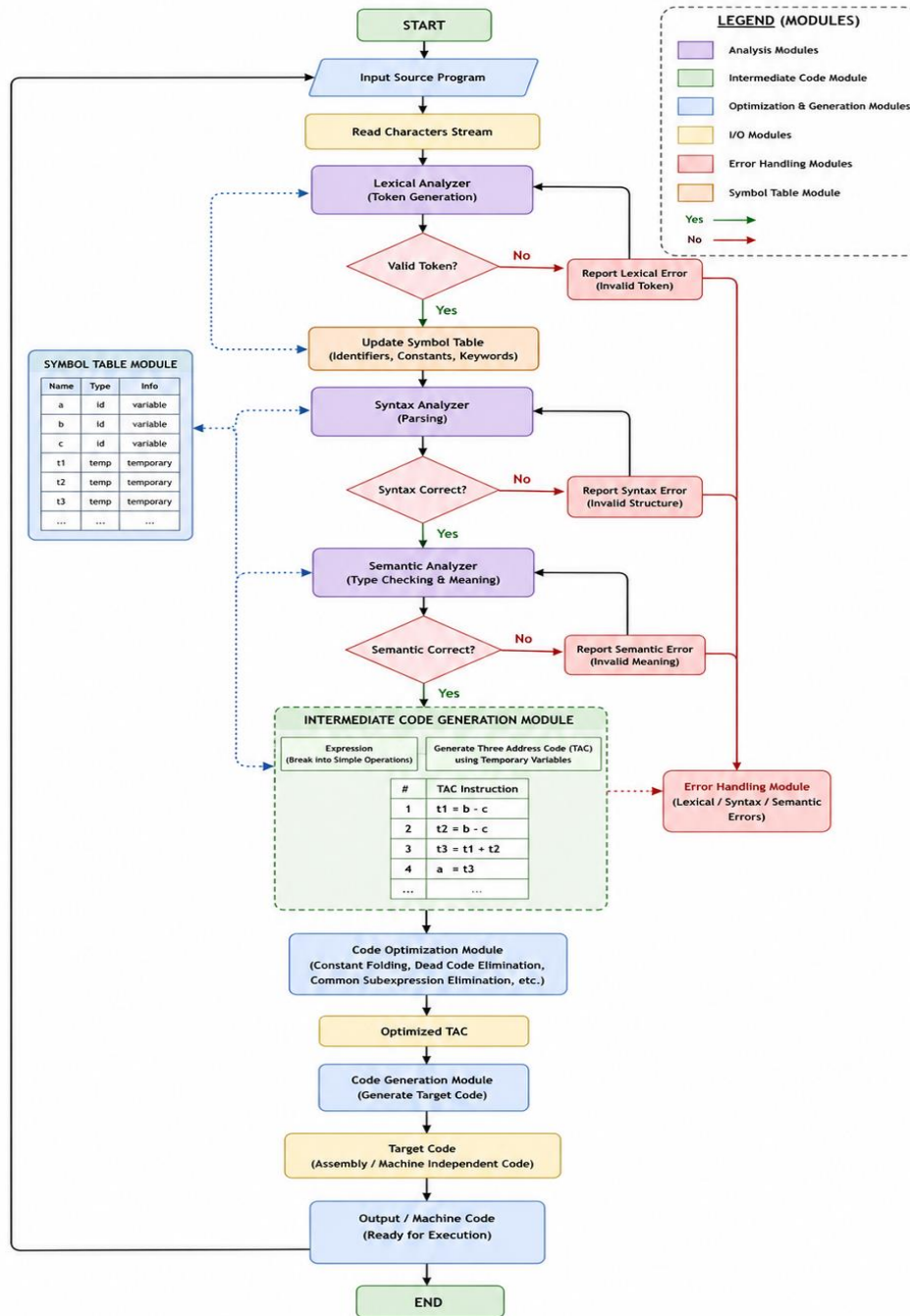
Machine Code

Compiler Workflow

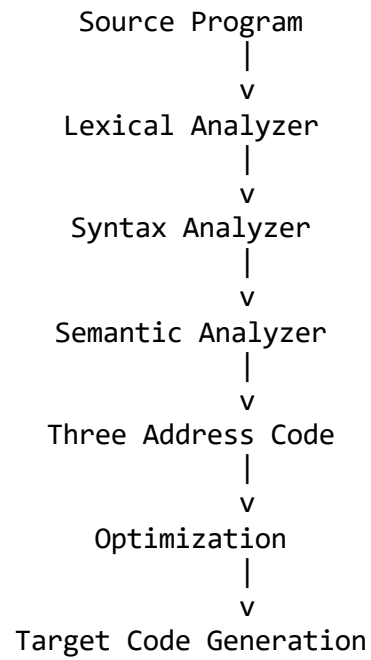


Flowchart:

THREE ADDRESS CODE (TAC) GENERATION FLOWCHART

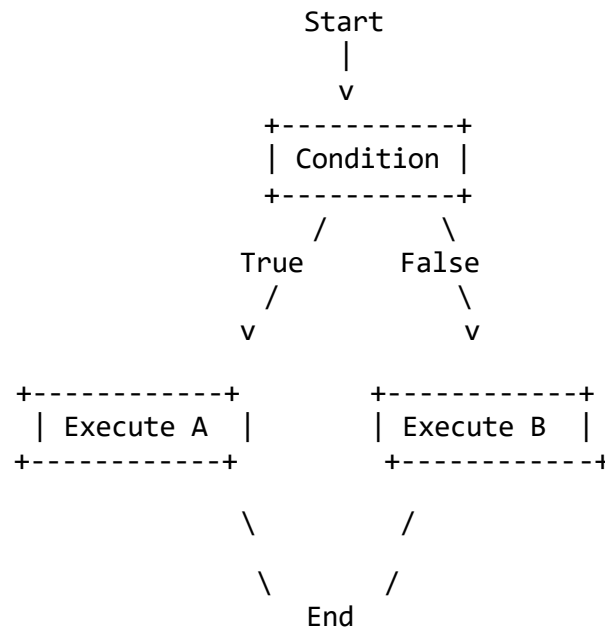


Processing Diagram



Conditional Diagram

Conditional statements in TAC are represented using branching instructions.



Representations of TAC

1. Quadruples

A quadruple contains four fields:

Operator	Argument1	Argument2	Result
+	b	c	t1

Advantages

- Easy implementation
- Suitable for optimization

2. Triples

A triple stores:

Index	Operator	Operand1	Operand2
-------	----------	----------	----------

Index	Operator	Operand1	Operand2
0	+	b	C

Advantages

- Requires less memory
- Efficient storage

3. Indirect Triples

Indirect triples use pointers to triples rather than storing instructions directly.

Advantages

- Easy code movement
- Better optimization support

Advantages of TAC

- Easy to understand and debug.
- Simplifies complex expressions.
- Supports compiler optimization.
- Machine-independent representation.
- Improves target code generation.
- Reduces compilation complexity.

Disadvantages of TAC

- Requires temporary variables.
- Intermediate code size increases.
- Additional memory consumption.
- Complex management for large programs.

Applications of TAC

- Compiler design and construction.
- Code optimization.
- Intermediate code generation.
- Program analysis.
- Educational compiler projects.

Conclusion

Three Address Code is one of the most important intermediate representations in compiler construction. It simplifies complex source code expressions into smaller instructions that are easier to optimize and translate into machine code. TAC improves compiler efficiency, supports advanced optimization techniques, and serves as a bridge between source code and target machine code.